Source: docs\POLICY_PAGE_DESIGN.md | Export: 11 November 2026

Storefront policy pages

Source and routes

The four existing public pages share `src/policies/PolicyLayout.tsx`. Their source remains the existing `siteSettings/storeContent` document, read through `/api/catalog?content=store` and the existing `StoreContentProvider`.

Canonical page	Route	CMS field	Admin editor
Privacy Policy	`/privacy-policy`	`policies.privacy.{heading,text,approved}`	`/admin/website-content/privacy`
Terms & Conditions	`/terms-conditions`	`policies.terms.{heading,text,approved}`	`/admin/website-content/terms`
Shipping Policy	`/shipping-policy`	`policies.shipping.{heading,text,approved}`	`/admin/website-content/shipping`
Returns & Exchanges	`/returns-policy`	`policies.returns.{heading,text,approved}`	`/admin/website-content/returns`

`/policies?tab=privacy|terms|shipping|returns` remains compatible through `normalizePath`. Existing footer, drawer, product, account and checkout links remain unchanged. Policy navigation now changes the actual route rather than only local tab state, so direct entry, refresh, Back and Forward agree. Canonical page titles use the existing `Seo` labels. Saved headings affect the h1 without renaming established routes or directory labels.

The public server projection already strips unapproved policy bodies and headings. The presentation additionally requires `approved === true` and nonblank text before rendering a policy. It does not infer approval from the presence of text. Valid legacy policies lacking a heading retain their canonical title. No draft text, private metadata or editor timestamps are requested separately or exposed by this change.

Layout and safe text

Desktop uses a controlled 1320px page width, compact 260px left directory, a reading column capped at 830px and a closing support panel. Navigation sticks below the measured shared header, including changes in header height when Search opens. Its own overflow remains scrollable on short screens. Mobile uses the real page selector, single-column reading, optional native collapsible section index and sufficient space for the existing five-tab navigation. No policy page activates Account. Floating WhatsApp is omitted on these pages so it cannot cover reading controls; existing source-driven support links and Contact Us remain available.

The shared header, logo, footer, local Lora, maroon `#780D24`, warm ivory `#FBF7F0` and gold accents are preserved. Standalone controls have 44px minimum targets. Text, long headings and unbroken URLs wrap. CSS is scoped under `.policy-page` and `.policy-*`; no zoom or scale is used. No remote fonts, dependencies or icon-font names are added.

The actual CMS format is **plain text**, up to 12000 JavaScript string code units, with an optional heading up to 100 characters. There is no structured heading/list/link array, rich text or policy-date field. Published content is rendered as escaped React text with `white-space:pre-wrap`; original paragraph breaks, literal bullets and URLs remain intact. HTML is never interpreted and URLs are not guessed into links. A section index is deliberately omitted for real plain text rather than inferring headings from sentences or changing the CMS format.

The separate local design fixture has explicit structured sections. Its index is generated from those actual fixture headings; position-based policy-prefixed IDs remain stable and unique even for duplicate headings. Links move focus and scroll below the measured header. Direct anchor refresh is supported. Reduced motion uses instant scrolling. Back to Top restores heading focus and removes the old section hash. Native details elements provide expanded/collapsed semantics on mobile.

Local sample review

Samples are in the separate `src/policies/PolicySample.tsx` module. They contain layout-review placeholders, not prototype business promises. They are never saved to Firestore, submitted through an Admin action or used as ordinary production fallback content.

Use the existing Vite development environment and local API setup, then visit one of:

```
/privacy-policy?preview=policy
/terms-conditions?preview=policy
```

```
/shipping-policy?preview=policy
/returns-policy?preview=policy
```

For example, start the existing local API with `npm.cmd run dev:api` and Vite with `npm.cmd run dev`. The public API must successfully load before a missing-policy sample is shown. An API failure shows Retry, never a substitute sample. Use isolated API/browser fixtures for reviews that must not read hosted content. The browser regression provides these read-only response fixtures without seeding CMS records.

The flag is checked against `import.meta.env.DEV`; it is not a public preview API. Samples appear only for a policy without usable approved public text. Existing approved text always wins, even when the preview flag is present. Preview navigation carries the flag between canonical policy routes.

Every displayed sample includes exactly:

> Sample content for design review. Final policy wording will be provided by Parinaya Silks.

Vite removes the sample import branch and its fixture module from normal production builds. Browser checks verify that a crafted production `?preview=policy` URL still shows the unavailable state and cannot load sample code. The production JavaScript artifact is checked for absence of the fixture marker.

Loading and failures

The pages distinguish initial loading, approved content, explicit local sample review, unavailable content, and request failure with Retry. They reuse the existing shared request and refresh interval without adding per-policy or per-section requests. A failed background refresh preserves previously loaded approved wording with an explicit refresh-error message. Unavailable production content displays the existing honest pending-terms sentence and Contact Us action.

There is no independent active-policy state that can disagree with the URL. All policies come from one public snapshot; the existing provider's abort/sequence guards reject older responses. A delayed earlier snapshot cannot overwrite a newer response or select a different policy.

Final client wording and release

The client still needs to provide and approve the final wording for all four policies. No policy content or approval state has been changed by this implementation. The owner enters the page heading and Main policy text in the corresponding existing Admin Website Content editor and explicitly confirms the existing approval control before saving. Published text appears through the existing no-store public refresh behavior (normally within 15 seconds or on refresh/revisit). Do not put unreviewed sample content into those editors.

There is no structured effective-date or version field in this contract, so prototype dates, versions, registry IDs and legal-review badges are omitted. Real date wording may be part of the owner's supplied plain text; this task adds no date schema or approval workflow.

Returns & Exchanges retains its actual name. The design's added Refunds/Cancellation labels do not create a new document, request flow or entitlement. Operational `siteSettings/returns` settings, shipping calculations, customer consent, checkout, payment methods, orders and loyalty are unchanged. No mandatory acceptance, cookie platform, PDF, print or sharing controls are introduced.

The owner needs only the normal frontend application release. There are no new environment variables, dependencies, server functions, migrations, Firestore indexes or rules. Hosted acceptance remains unverified until the owner releases and checks the pages against the real approved content. No commit, push, deployment, CMS seed, rules/index publication or production-data change was performed.

Verification artifacts

The local unit/API/browser results and final visual review are recorded after execution below. New coverage is in `src/policies/model.test.tsx` and `scripts/policy-design-browser.test.py`. Existing `scripts/website-content-emulator.test.ts` and `scripts/website-content-browser.test.py` cover actual Admin save/public publication, approval and private-draft exclusion, both administrator requirements, shared-field preservation, Unicode, private previews and retry/conflict handling.

Screenshots are ignored under `.cache/policy-design/`:

- `{privacy,terms,shipping,returns}-{320,390,768,1440,1920}.png`
- `published-long-1920.png`
- `stitch-{privacy,terms,shipping,returns}-{390,1440}.png`
- `comparison-{privacy,terms,shipping,returns}-{390,1440}.html` and `.png`

Each original desktop/mobile export is rendered at the matching 1440px/390px CSS width with its styles loaded. Some reference captures retain prototype controls; none are included in the application. Local captures use explicitly labelled, isolated preview fixtures. Actual approved content and missing/error states are tested separately. The existing shared shell/Lora, canonical Returns & Exchanges label, omitted unsupported claims/dates and lack of a guessed real-content section index are deliberate differences from Stitch.

Completed local verification

- `npm.cmd run lint`: TypeScript check passed.
- `npm.cmd test`: 152 tests passed, 0 failed, including the three new policy tests and existing routing, content, catalog, customer and administrator unit regressions.
- `npm.cmd run build`: production build passed. The existing large frontend-chunk advisory remains; no new build error was introduced.
- `scripts/website-content-emulator.test.ts`: passed against isolated Auth/Firestore emulators. Actual publication, private projection, approval and both administrator requirements remain protected.
- `scripts/website-content-browser.test.py`: passed against isolated local services. Existing Admin save/public rendering, shared fields, Unicode, conflict and retry behavior were exercised.
- `scripts/policy-design-browser.test.py`: passed after the final presentation changes. All four pages were captured at 320, 390, 768, 1440 and 1920px with no horizontal page overflow, local Lora and no falsely active Account tab. Coverage includes navigation/history/aliases, keyboard section links and direct anchor refresh, approved content over samples, private drafts, short and long plain text, long headings/URLs, loading/error/retry, retained content and delayed stale responses.
- The built production app was exercised separately: crafted sample flags do not render/load sample fixtures, missing content stays unavailable, and approved policy content renders correctly. The sample marker is absent from built JavaScript.
- `git diff --check`: passed.

All eight desktop/mobile reference comparisons were visually reviewed at matching CSS widths after the final captures. Desktop framing, reading-width limits, informational cards, mobile selection/index and footer transition were checked. Differences necessary to preserve the shared storefront, accessible targets, actual canonical names, plain-text CMS format and truthful placeholder copy are described above. No pixel-perfect equivalence is claimed for unsupported prototype content or its separate shell.

Logs are ignored under `.cache/policy-unit.log`, `.cache/policy-regressions.log`, `.cache/policy-final-build.log` and `.cache/policy.log`. Tests did not open support applications. Emulator services were shut down after completion. These results establish local source/build behavior, not hosted production acceptance or approval of policy wording.